

河南大学 2026 届本科毕业论文

基于 GraphRAG 的教学智能问答系统的
设计与实现

Design and Implementation of an Intelligent Teaching Q&A
System Based on GraphRAG

论文作者姓名：_____ 韩宁 _____

作 者 学 号：_____ 2212080182 _____

所 在 学 院：_____ 计算机与信息工程学院 _____

所 学 专 业：_____ 计算机科学与技术 _____

导 师 姓 名：_____ 葛强 _____

导 师 职 称：_____ 教授 _____

2026 年 4 月 20 日

摘 要

随着教育信息化 2.0 持续推进，高校教学中的知识管理与智能检索面临新挑战：传统教材以线性文本组织知识，学生难以快握概念间的关联结构；现有检索方式依赖关键词匹配或向量相似度计算，无法有效支撑因果、依赖关系及多步推理任务。检索增强生成（RAG）通过外部知识库增强大语言模型的生成能力，缓解了幻觉与知识滞后问题，但基于文档分块的向量检索在语义割裂、实体关系捕捉和多跳推理方面仍有固有不足。GraphRAG 将知识图谱引入 RAG 流程，以结构化图谱组织知识，使关联推理和多跳问答成为可能，为教育领域的知识管理与智能问答提供了新的解决思路。

本文在腾讯优图实验室 Youtu-GraphRAG 开源框架的基础上进行二次开发，设计并实现了一个基于 GraphRAG 的知识图谱智能问答系统。主要工作为设计 Schema 引导的知识抽取流程，从 PDF、Word 教材自动构建知识图谱，实现章节识别、实体对齐、增量合并；实现双路并行混合检索策略，采用 FAISS 向量检索和 Neo4j 图遍历，优先使用 GraphQ 问题分解机制支持复杂问题的多子问题并行检索，在不可用时回退为单问题检索；在原有单体后端基础上增加 FastAPI 微服务层，使用 Celery 异步任务队列管理构建流程，整合 Neo4j 图数据库、Redis 缓存、MinIO 对象存储，实现前后端分离部署；基于 Vue 3 开发可视化前端界面，支持图谱展示、智能问答、图谱编辑。系统已经通过功能测试，可以进行实际部署。

关键词：知识图谱；GraphRAG；智能问答；微服务架构；检索增强生成

ABSTRACT

With the advancement of Education Informatization 2.0, knowledge management and intelligent retrieval in higher education face new challenges: linear textbook structures hinder students from grasping conceptual relations, while conventional retrieval based on keywords or vector similarity fails to support causal reasoning and multi-step inference. Retrieval-Augmented Generation (RAG) mitigates hallucination and knowledge staleness by leveraging external knowledge bases, yet its document-chunking approach still suffers from semantic fragmentation, weak entity-relation capture, and limited multi-hop reasoning. GraphRAG addresses these limitations by introducing knowledge graphs into the RAG framework, organizing knowledge as structured graphs that enable relational reasoning and multi-hop question answering, offering a novel solution for educational knowledge management and intelligent tutoring.

Based on the open-source Youtu-GraphRAG framework from Tencent Youtu Lab, this paper designs and implements a GraphRAG-based knowledge graph question answering system. It proposes a schema-guided knowledge extraction pipeline to automatically construct knowledge graphs from PDF and Word textbooks, supporting chapter recognition, entity alignment and incremental merging. A dual-path hybrid retrieval strategy combining FAISS vector retrieval and Neo4j graph traversal is adopted, which uses GraphQ query decomposition for parallel sub-query retrieval and falls back to single-question retrieval when necessary. The system employs a FastAPI-based microservice architecture with Celery for asynchronous task scheduling, and integrates Neo4j, Redis and MinIO to realize separated front-end and back-end deployment. A visualized front-end interface is developed with Vue 3, supporting graph display, intelligent question answering and graph editing. Functional tests verify that the system can be deployed in practical applications.

Keywords: Knowledge Graph; GraphRAG; Intelligent Question Answering; Microservice Architecture; Retrieval-Augmented Generation

目 录

摘 要.....	I
ABSTRACT.....	II
第 1 章 绪 论.....	1
1.1 课题来源.....	1
1.2 课题背景.....	1
1.3 国内外研究现状及分析.....	1
1.3.1 知识图谱构建技术研究现状.....	1
1.3.2 检索增强生成技术研究现状.....	2
1.3.3 GraphRAG 技术研究现状.....	2
1.4 本文主要研究内容与论文结构.....	3
第 2 章 相关技术介绍.....	4
2.1 知识图谱基础理论.....	4
2.1.1 知识图谱的定义与表示.....	4
2.1.2 知识抽取技术.....	4
2.2 大语言模型与检索增强生成.....	5
2.2.1 大语言模型概述.....	5
2.2.2 RAG 技术原理与局限性.....	5
2.2.3 GraphRAG 技术原理.....	6
2.3 系统开发相关技术.....	6
2.3.1 Neo4j 图数据库与 Cypher 查询语言.....	6
2.3.2 FAISS 向量检索技术.....	6
2.3.3 FastAPI 框架与微服务架构.....	7
2.3.4 Vue.js 前端框架.....	7
2.4 本章小结.....	7
第 3 章 GraphRAG 教学智能问答系统需求分析.....	8
3.1 需求定位.....	8
3.2 系统需求分析.....	8
3.2.1 功能需求分析.....	8
3.2.2 非功能需求分析.....	9
3.3 系统用例分析.....	9
3.3.1 教师用例.....	10
3.3.2 学生用例.....	11
3.4 本章小结.....	12
第 4 章 GraphRAG 教学智能问答系统设计.....	13
4.1 系统总体架构设计.....	13
4.1.1 系统架构设计.....	13
4.1.2 技术选型与对比.....	14
4.2 数据存储设计.....	15
4.2.1 Neo4j 图模型设计.....	15
4.2.2 Redis 缓存策略设计.....	15
4.2.3 MinIO 文件存储设计.....	16

4.3 功能模块设计	16
4.3.1 知识图谱自动构建模块设计	16
4.3.2 智能检索与问答模块设计	17
4.3.3 图谱展示与编辑模块设计	17
4.4 本章小结	18
第 5 章 GraphRAG 教学智能问答系统实现与展示	19
5.1 知识图谱自动构建模块实现与展示	19
5.1.1 Schema 引导的知识抽取实现	19
5.1.2 文档解析与章节识别实现	20
5.1.3 实体对齐与增量合并实现	21
5.1.4 异步构建任务管理实现	21
5.1.5 课程管理与自动构建功能展示	22
5.2 智能检索与问答模块实现与展示	23
5.2.1 双路并行混合检索实现	23
5.2.2 GraphQ 问题分解实现	24
5.2.3 多轮对话会话管理	24
5.2.4 问答交互与推理路径展示	25
5.3 图谱展示与编辑模块实现与展示	25
5.3.1 图谱可视化展示实现	25
5.3.2 图谱编辑与索引更新实现	26
5.3.3 图谱编辑功能展示	27
5.4 后端微服务与部署实现	27
5.4.1 API 模块化路由设计与统一响应	27
5.4.2 Docker Compose 一键部署	28
5.5 本章小结	29
结 论	30
参考文献	32
致 谢	33

第1章 绪论

1.1 课题来源

课题来源为 2025-2027 年“人工智能+教育”领航计划。

1.2 课题背景

在高等教育的环境下，教材是重要的知识传递工具，它具有很强的线性特点，使得学生很难对知识点之间的联系有深刻的认识。以中国古典文学课程为例，水浒传里“高俅权倾朝野”、“林冲被陷害”以及“鲁智深惩凶救弱”这些故事互相关联、错综复杂地构成了多种人物之间的相互影响和多方面叙述线索并存的局面。目前主流的信息检索工具大多只停留在对文本进行简单的表面搜索上，不能进行深入的因果关系研究，对于“林冲为什么最终走上反抗命运的道路”的分析就是其中的一种。这类问题一般会打破章节或者回目内部的局部内容范围，对学生的综合思维水平提出较高的要求。

近些年来，大语言模型（Large Language Model, LLM）飞速发展，催生出许多智能问答方面的革新，其中包含检索加强生成（Retrieval-Augmented Generation, RAG）架构，该架构借助外部知识库达成信息融合以及表现力的明显改善，而且较好地解决了模型出现的偏差以及数据时效性的难题^[1]。传统的依靠文档分段的 RAG 系统对于长文本处理的时候，会出现缺少语义连贯性、缺少实体关联、层次逻辑推理能力差等诸多难题。为了克服这些不足，GraphRAG 的方法使用结构化的图模型来改善知识表示的形式，用图形化的知识网络支持关系推理，解决复杂的多步问题^[2]。

1.3 国内外研究现状及分析

1.3.1 知识图谱构建技术研究现状

知识图谱的演变过程可以分成两个主要部分，即手工构建和自动化产生。早期的以 DBpedia、YAGO 为代表的典型实例都是用人工编写而成的规则以及结构化数据源来构建起来的。伴随着深度学习技术的发展，远程监督的方法以及联合标注策略也渐渐被运用到命名实体识别、关系提取等任务的研究当中。近些年来，以大规模预训练语言模型

（LLM）为依托，依靠先进的 LLM 技术，学者们对非结构化文本里的人、事、物及其联系进行了大量的自动化获取工作^[3]。唐晓晟等人^[4]尝试利用大语言模型自动构建学科知识图谱，验证了 LLM 在教育领域知识抽取中的可行性。不过，现有方法在面向特定领域时仍存在 Schema 定义困难、实体对齐精度不够等问题。

1.3.2 检索增强生成技术研究现状

2020 年，Lewis 等人^[1]第一次提出了 RAG（检索增强生成）框架，通过检索外部知识库增强语言模型的生成能力。Gao 等人^[5]对 RAG 技术做了全面综述，将其演进划分为朴素 RAG、高级 RAG 和模块化 RAG 三个阶段。尽管 RAG 缓解了 LLM 的幻觉问题，但是它的依靠文档切分的传统向量索引方式仍然存在着明显的不足，因为片段化的操作很容易造成语义相关的知识被遗漏，仅仅依靠余弦相似度计算很难准确体现实体之间的结构联系，而对于那些需要进行多阶段逻辑推理的任务来说，单次的检索过程一般不能形成完备的证据链条。

1.3.3 GraphRAG 技术研究现状

为克服传统 RAG 的局限性，研究者将知识图谱与 RAG 相结合，提出了 GraphRAG 技术。Microsoft 于 2024 年提出 GraphRAG 方法^[6]，它的基本想法就是用大规模预训练语言模型来构建实体间的关系知识网络，再利用社区级摘要来辅助全局问题求解，但是它存在很高的构建成本。LightRAG^[7]采用双层检索策略，在检索效率和准确性之间取得了较好的平衡。GRAG^[8]关注文本子图的检索与生成，提出了分治策略来优化子图提取。腾讯优图实验室提出的 Youtu-GraphRAG^[1]通过 Graph Schema 统一约束图构建和图检索两个阶段，实现了构建成本的降低和准确率的提升。Han 等人^[2]对 GraphRAG 技术做了系统性综述，构建了包含查询处理器、检索器、组织器、生成器和数据源的统一框架。主流框架如表 1.1 所示。

表 1.1 主流 GraphRAG 框架对比

框架	核心思路	构建成本	推理能力	适用场景
Microsoft GraphRAG ^[6]	社区分层摘要	高	全局问答	大规模语料全局理解
LightRAG ^[7]	双层图检索	中	局部+全局	通用知识问答
GRAG ^[8]	文本子图检索	中	子图推理	图结构数据问答
Youtu-GraphRAG ^[9]	Schema 约束	低	多跳推理	垂直领域深度推理

本研究所选 Youtu-GraphRAG 二次开发平台的主要原因是因为它依靠 Schema 来驱动，可以很好地满足教育领域内结构化知识提取的技术要求，双通道并行检索架构在语义匹配准确率以及实体关系推断性能上具有明显的优势，研发成本低，是资源有限的教学环境中的最佳选择。

1.4 本文主要研究内容与论文结构

本文是在 Youtu-GraphRAG 的基础上进行进一步改进优化，主要工作有以下几个方面。

（1）提出一种基于 Schema 的智能化知识抽取方法，用于教材 PDF、Word 等文件的自动生成以及一个具有章节解析、实体映射、增量更新等功能的知识图谱生成工具。

（2）使用双通道并行混合检索方式结合 FAISS 向量索引以及 Neo4j 图数据库查询方法并且利用 GraphQ 的问题拆分能力进行复杂的多子问题并行检索及推断。

（3）基于原有单体后端，在其之上增加了一层 FastAPI 微服务，使用 Celery 异步任务队列进行调度，使用 Neo4j 图数据库、Redis 缓存以及 MinIO 对象存储。

（4）基于 Vue3 实现可视化前端页面，能够显示图谱、进行智能问答以及编辑图谱等。

本文共分为五章。第二章介绍知识图谱、大语言模型、系统开发相关技术；第三章对系统进行需求分析，从功能需求和非功能需求两个方面对系统进行全面分析；第四章进行系统设计，包括总体架构、数据存储、功能模块设计；第五章详细阐述各个核心模块的实现及功能展示，并对系统进行测试验证；最后对全文进行总结并展望未来工作。

第 2 章 相关技术介绍

本章对系统涉及的关键技术进行介绍，包括知识图谱基础理论、大语言模型与检索增强生成技术，以及系统开发所依赖的框架和工具。

2.1 知识图谱基础理论

2.1.1 知识图谱的定义与表示

知识图谱（Knowledge Graph, KG）是一种以图结构组织和表达知识的语义网络，其基本组成单元为三元组（头实体，关系，尾实体），记作 (h,r,t) 。其中，实体表示现实世界中的概念或对象，关系描述实体之间的语义关联。例如，在中国古典文学领域，三元组（林冲，属于，八十万禁军）表达了人物“林冲”与组织“八十万禁军”之间的从属关系^[10]。

知识图谱的存储框架主要是分属性图模型和 RDF 三元组模型两种。本文使用属性图模型，并用 neo4j 图数据库进行数据管理。此模型以节点、边都可以承载属性键值对为明显特点，在保存实体的多种元数据信息（种类、可靠程度、出处等）上具有较好的效果，特别适合于动态变化并且结构比较复杂的知识领域数据存储。

2.1.2 知识抽取技术

知识抽取技术就是从非结构化或者半结构化的文本中提取出实体、关系和属性信息的过程，也是形成知识图谱的一个过程。传统的知识抽取方法包括用规则的方法进行模式匹配的、用 BiLSTM-CRF 序列标注的、用深度学习的方法等几种类型。虽然这些方法在一定的应用场合下取得了较好的效果，但是它在标注数据量大、泛化能力弱等缺点上还比较明显^[3]。

近些年来，以大语言模型为主的知识抽取技术渐渐成为了研究领域的主流范式。通过精心设计好的提示词（Prompt）来使这种模型可以直接从非结构化的文本当中得到需要的三元组数据，大幅度减少了对人工标注数据的使用^[4]。本系统采用 Schema 引导的抽取策略，即先定义好实体类型、关系类型、属性类型等 JSON Schema，约束 LLM 的抽取输出格式，保证抽取结果的一致性、可控性。

表 2.1 知识抽取方法对比

方法	代表技术	优点	局限性
规则匹配	正则表达式、词典	精度高	覆盖率低，需人工维护
统计学习	CRF、HMM	可自动学习特征	依赖标注数据
深度学习	BiLSTM-CRF、BERT	泛化能力强	需大量标注数据
LLM 驱动	GPT、DeepSeek + Prompt	零样本/少样本抽取	需 Schema 约束保证一致性

2.2 大语言模型与检索增强生成

2.2.1 大语言模型概述

大语言模型（Large Language Model, LLM）是基于 Transformer 架构^[1]的大规模预训练语言模型。Transformer 依靠自注意力机制来处理序列里复杂的长距离依赖关系，突破了传统递归神经网络的线性运算限制，达成高效率并行训练的目的。研究表明，以 GPT 系列、DeepSeek 为代表的一些典型案例表明，预训练后的这些模型对于大量的文本数据集来说，已经系统地习得了各种语言表达方式以及语义解析的能力，具有了非常高的生成流畅且有表现力自然语言文本的潜力。

本系统通过统一的 LLM 调用接口（utils/call_llm_api.py）支持 DeepSeek API 和本地 Ollama 模型两种调用方式，在知识抽取、问题分解和答案生成三个环节中均依赖 LLM 的能力。

2.2.2 RAG 技术原理与局限性

检索增强生成（Retrieval-Augmented Generation, RAG）技术由 Lewis 等人^[1]于 2020 年提出，基本思路是将信息检索和自然语言处理领域中的文本生成任务融合起来，即用户提出问题后，在外部知识库中找到对应的知识点并提取出相关文献的片段或者语义单位，将其当作补充的背景信息提供给预先训练好的大型语言模型，引导模型生成基于事实的答案。RAG 缓解了 LLM 的幻觉问题和知识过时问题^[5]。

但是传统 RAG 依靠文档分块的向量检索存在三个固有的局限：

（1）语义割裂：文档按固定长度切分为独立块，跨块的语义关联在切分过程中被破坏。例如，“林冲误入白虎堂”的完整描述可能被分散在两个相邻块中。

（2）缺乏结构化关系：向量相似度检索只能找到语义相近的内容，无法捕捉概念之

间的因果、依赖、包含等结构化关系。

（3）不支持多跳推理：对于需要经过多步推理的复杂问题，单次检索很难获取完整的证据链。

2.2.3 GraphRAG 技术原理

GraphRAG 是将知识图谱与 RAG 相结合的技术范式^[2]。基本思路是：先利用 LLM 从原始文档中抽取实体和关系，构建知识图谱；检索阶段通过图遍历和结构化查询获取关联知识，而不是只用向量相似度。

GraphRAG 相比传统 RAG 的改进在于：（1）知识图谱以三元组形式保存实体间的结构化关系，避免了语义割裂；（2）图遍历可以沿着关系路径发现隐含的关联知识；（3）多跳推理能够沿着推理链逐步扩展证据。以 Youtu-GraphRAG^[9]为代表的框架进一步引入 Schema 约束和分层知识树，在构建成本和推理精度上取得了较好的效果。

2.3 系统开发相关技术

2.3.1 Neo4j 图数据库与 Cypher 查询语言

Neo4j 是目前使用最广泛的原生图数据库，使用属性图模型存储数据，节点和关系都可以携带属性。相比关系型数据库，Neo4j 在多跳关联查询上更具有优势，关系型数据库需要进行多表 JOIN 操作，查询复杂度随着跳数的增加而指数增长，而 Neo4j 通过图遍历直接沿着关系边访问相邻节点，查询复杂度与结果集大小成正比。

本系统使用 Neo4j 存储知识图谱的实体节点和关系边，并通过 Cypher 查询语言实现节点检索、邻域扩展、子图提取和路径搜索等操作。每个实体节点携带 name、type、chapter、confidence 等属性，支持按章节过滤和按置信度筛选。

2.3.2 FAISS 向量检索技术

FAISS（Facebook AI Similarity Search）是由 Meta 开源的高效向量相似度检索库^[12]，支持十亿级向量的快速检索。FAISS 提供了多种索引类型，本系统使用 IndexFlatIP（内积索引），通过对向量进行 L2 归一化后计算内积，等价于余弦相似度检索。

本系统构建了四个独立的 FAISS 索引：node.index（节点向量）、relation.index（关系向量）、triple.index（三元组向量）和 comm.index（社区摘要向量），分别服务于不

同的检索场景。在双路并行检索策略中，`node.index` 和 `triple.index` 分别支撑两条检索路径的向量相似度计算。

2.3.3 FastAPI 框架与微服务架构

FastAPI 是一个基于 Python 的现代 Web 框架，具备原生异步支持、自动 API 文档生成（OpenAPI/Swagger）和基于 Pydantic 的类型安全校验。相较于 Flask，FastAPI 基于 ASGI 协议支持异步 IO，适合处理 LLM 调用等高延迟请求；相较于 Django，FastAPI 更轻量，适合构建微服务。

本系统在原有单体后端（`backend.py`）基础上新增 FastAPI 微服务层，将路由拆分为 `courses`、`documents`、`graph`、`graph_edit`、`qa`、`tasks`、`schemas`、`auth`、`health` 共 9 个独立模块，通过统一响应格式（`{code, message, data}`）和全局异常处理保证 API 的一致性。异步任务通过 Celery+Redis 实现，支持知识图谱构建等长时间运行任务的非阻塞执行。

2.3.4 Vue.js 前端框架

Vue.js 是一个渐进式 JavaScript 前端框架，使用组件化开发模式。本系统前端基于 Vue 3 的 Composition API 构建，使用 Vite 作为构建工具。图谱可视化使用 ECharts 图表库，采用 force 力导向布局来展示知识图谱的网络关系，支持节点拖拽、缩放、分类着色、详情抽屉等交互功能。对大规模的课程图谱，前端配合后端 `scope` 参数来实现分层展示策略，先展示骨架图（`atlas`）供快速概览课程整体结构，在此基础上可以切换概览图（`overview`）和完整图（`full`）；对完整图的加载过程采用异步加载和渐进式渲染的方式，以减少浏览器端大图的渲染卡顿。UI 组件库选择 Element Plus，提供表单、表格、对话框等标准组件。前端通过 Axios 发起 HTTP 请求，与后端 FastAPI 服务进行 RESTful API 通信，实现前后端分离架构。

2.4 本章小结

本章介绍了系统涉及的四类技术：知识图谱的基本概念和 LLM 驱动的知识抽取方法；RAG 技术的原理及其局限性，以及 GraphRAG 如何通过引入知识图谱克服这些局限；Neo4j 图数据库、FAISS 向量检索库、FastAPI 框架和 Vue.js 前端框架等开发技术。这些技术构成了本系统的技术基础，后续章节将围绕它们展开系统的设计与实现。

第3章 GraphRAG 教学智能问答系统需求分析

本章从系统定位和目标出发，对系统的功能需求和非功能需求进行全方位的分析，用用例分析确定系统的用户角色和交互场景，为后续的系统设计提供依据。

3.1 需求定位

本系统面向高校教学场景，给教师和学生提供知识图谱构建、展示、问答、编辑四类主要功能。系统在腾讯优图实验室 Youtu-GraphRAG 开源框架的基础上进行二次开发，将原来的单体后端拆分为 FastAPI 微服务层，整合 Neo4j 图数据库、Redis 缓存、MinIO 对象存储，最终封装为独立微服务，可以对接智慧学习平台。

系统主要有两类用户：教师是知识图谱的创建者和管理者，教师上传教材文档、触发图谱构建、审核和编辑图谱内容。教师查看全量图谱数据（低置信度节点），发现抽取错误后手动修正。学生是知识图谱的使用者，学生通过浏览图谱、智能问答来学习课程知识。学生只能看到已经发布的高质量内容，防止低置信度的信息干扰学习。

3.2 系统需求分析

3.2.1 功能需求分析

本系统面向高校教学场景，为教师和学生提供知识图谱构建、展示、问答和编辑四类核心功能。通过分析教学实践中的实际需求，将系统功能需求整理如表 3.1 所示。

表 3.1 系统功能需求列表

功能模块	子功能	功能描述
课程管理	创建/删除课程	每门课程对应一个独立的知识图谱，课程间图谱完全隔离
文档管理	上传/删除文档	支持 PDF、DOCX、DOC、TXT、MD、RTF、JSON 格式，上传后自动触发图谱构建
图谱构建	自动构建	解析文档章节结构，调用 LLM 抽取实体和关系，写入 Neo4j
	进度查询	轮询构建任务状态（pending → processing → completed/failed）
	失败重试	按文档粒度重试失败的构建任务
图谱展示	教师视图	展示全量节点（含低置信度），显示来源文档、置信度等元信息
	学生视图	仅展示已发布的高质量节点，隐藏低置信度内容
	分层展示	支持骨架图（atlas）、概览图（overview）和完整图（full）三级展示策略
	章节子图	按章节属性过滤，展示指定章节的知识子图
智能问答	多轮对话	基于 Redis 存储会话上下文，支持连续追问
	推理路径	可视化展示问答过程中的图谱遍历路径
图谱编辑	增删节点/关系	支持最多 200 个 JSON 批量操作，逐条执行并返回结果汇总，新添加内容标记置信度
	索引更新	编辑后自动触发 FAISS 索引重建，保证检索一致性

3.2.2 非功能需求分析

除功能需求外，系统还需满足以下非功能需求：

表 3.2 系统非功能需求

需求类型	具体要求
性能	单次问答响应时间 3-5 秒（已构建图谱前提下），10 页 PDF 文档构建耗时 15 分钟以内
可靠性	构建任务失败可按文档粒度重试，服务异常时不丢失已完成的数据
可扩展性	课程间图谱隔离支持多租户，模块化 API 路由便于功能扩展
部署	Docker Compose 一键启动所有服务，健康检查接口支持容器编排

3.3 系统用例分析

根据以上需求分析，系统分为两类用户角色，每类用户对应不同的用例集合。

3.3.1 教师用例

教师是系统的管理者，拥有最高权限，功能包括创建和管理课程、上传和管理文档、查看知识图谱、编辑知识图谱、审核发布内容、按章节查看子图，如图 3.1 所示。

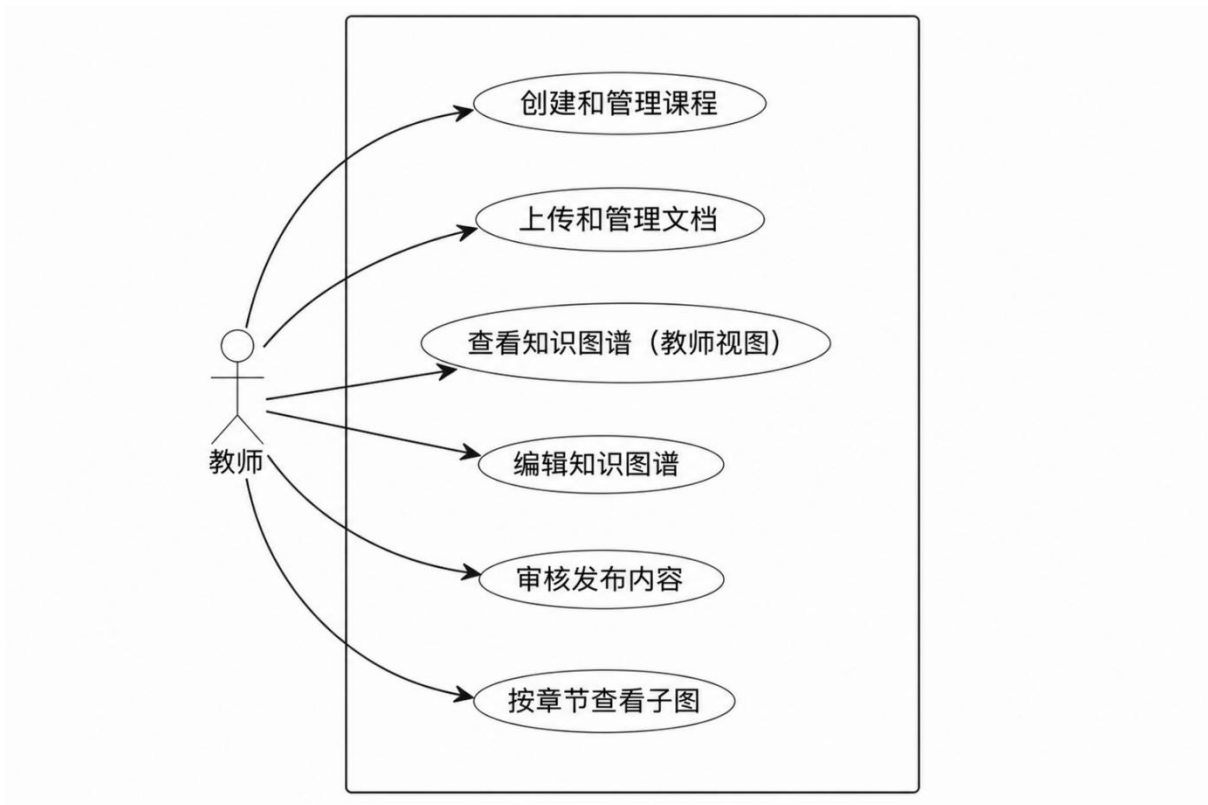


图 3.1 教师用例图

教师用例具体描述：

（1）创建和管理课程：创建新课程，删除不再需要的课程，查看课程列表和构建状态。

（2）上传和管理文档，上传 PDF、Word 等格式的教材文档，删除已上传的文档，查看文档构建进度。

（3）查看知识图谱，以教师视图查看全量图谱，包括低置信度节点和草稿状态节点，查看节点详情（来源文档、置信度、关联关系等）。

（4）编辑知识图谱，添加、修改、删除节点和关系，支持批量操作（最多 200 个 JSON 操作），合并同名实体，拆分错误合并的节点。

（5）审核发布内容：审核新添加或编辑后的节点，将草稿状态的内容发布为学生可见。

（6）按章节查看子图：选择特定章节，查看该章节的知识子图，辅助教学内容组

织。

3.3.2 学生用例

学生作为系统的使用者，功能包括浏览知识图谱，按章节学习、智能问答、多轮对话、查看推理路径，如图 3.2 所示。

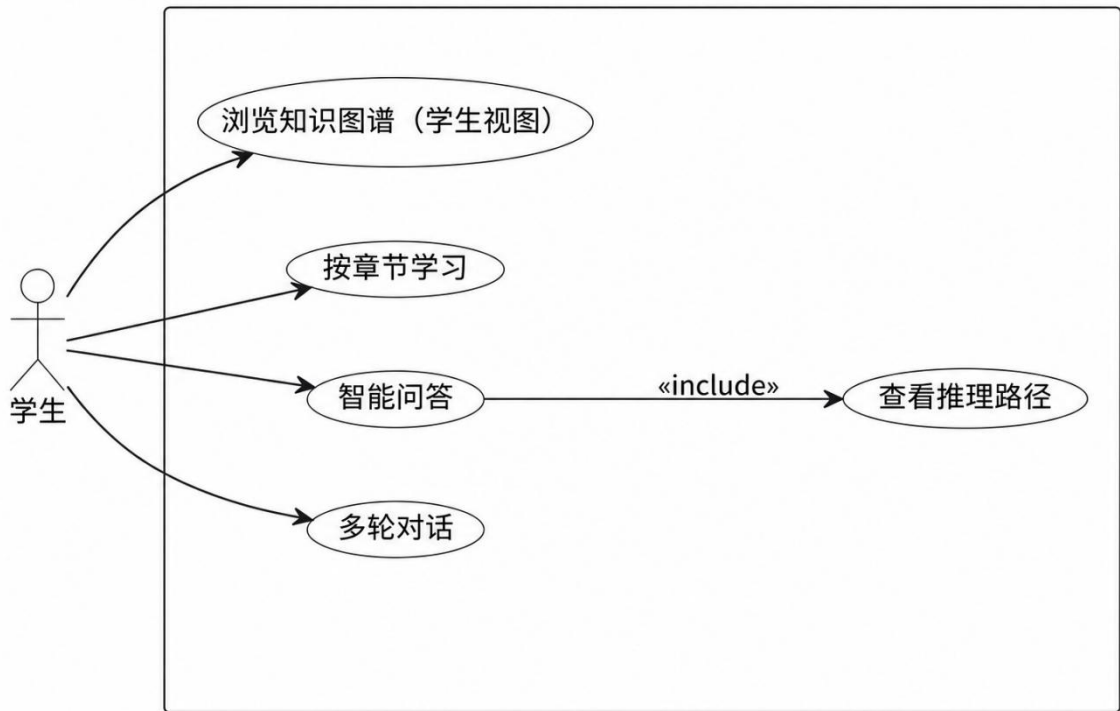


图 3.2 学生用例图

学生用例具体描述：

- (1) 浏览知识图谱：以学生视图查看已发布的知识图谱，仅展示高质量节点（ $\text{confidence} \geq 0.7$ 且 $\text{visibility} \neq \text{draft}$ ）。
- (2) 按章节学习：选择当前学习的章节，查看该章节的知识子图，聚焦学习内容。
- (3) 智能问答，输入问题，得到基于知识图谱的结构化回答，查看引用的三元组证据和推理路径。
- (4) 多轮对话：在已有会话中连续追问，系统自动携带历史上下文。
- (5) 查看推理路径，即在问答结果中查看系统推理过程的可视化子图，理解答案的来源和依据。

3.4 本章小结

本章从系统定位和目标出发，确定了系统面向高校教学场景的服务定位，分析了教师和学生两个用户角色的不同需求。在此基础上，从功能需求和非功能需求两个方面对系统进行了全面分析，整理出 13 项功能需求和 4 类非功能需求。用例分析得到教师和学生各自用例集合，为之后系统设计提供清晰的需求边界。

第 4 章 GraphRAG 教学智能问答系统设计

本章在需求分析的基础上，设计系统的总体架构、数据存储方案、功能模块和安全防护策略，为后面系统实现提供设计蓝图。

4.1 系统总体架构设计

4.1.1 系统架构设计

本系统采用前后端分离的四层架构设计，如图 4.1 所示。

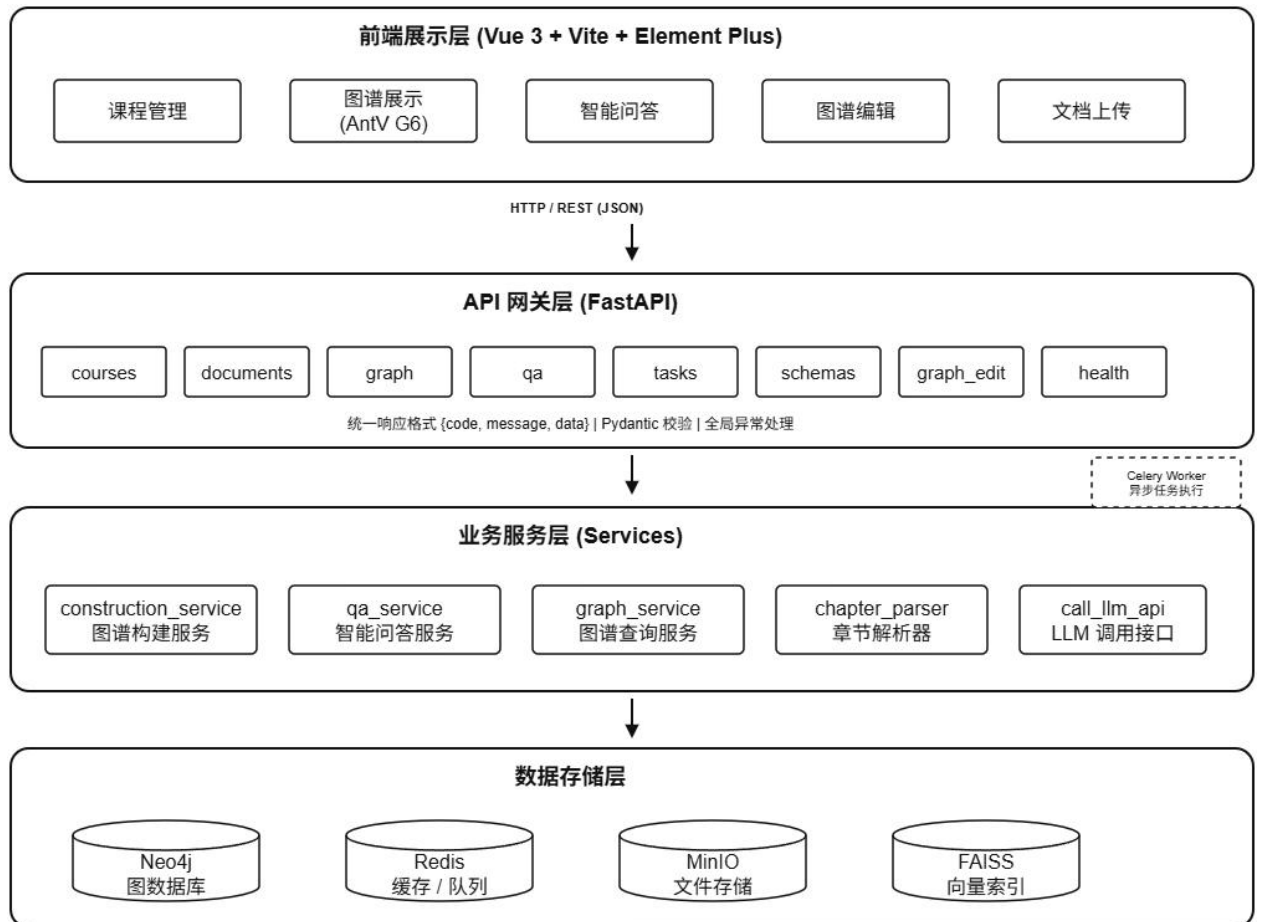


图 4.1 系统四层架构图

各层职责说明如下：

前端展示层：以 Vue 3 为基础开发单页应用，在其中集成 ECharts 图表库用于绘制知识图谱力导向布局，可进行节点拖拽、缩放、路径高亮、详情抽屉等功能操作，针对课程图谱规模不同，系统采用骨架图、概览图、完整图分层展示的方法，通过异步加载

以及渐进式渲染减少大图带来的页面闪烁问题。Element Plus 提供了表单、对话框、进度条等常用 UI 控件。

API 网关层：FastAPI 框架承载 9 个模块化路由，进行请求转发、参数校验、统一返回结果包装工作。每个路由只负责一个功能，`courses` 路由处理课程 CRUD 操作，`documents` 路由处理文件上传以及删除，`graph` 路由处理图谱查询，`graph_edit` 路由处理图谱编辑，`qa` 路由处理会话式智能问答，`tasks` 路由处理异步任务状态获取。全局 CORS 中间件可用于解决前后端分离问题。

业务服务层：封装核心业务逻辑。`construction_service` 负责调用 LLM 进行知识抽取并将结果写入 Neo4j；`qa_service` 实现双路并行检索以及问题拆解；`graph_service` 提供图谱查询及编辑功能；`chapter_parser` 负责对文档章节进行划分。

数据存储层：有多类存储引擎配合使用。Neo4j 用于存储知识图谱中的实体以及它们之间的关系；Redis 保存构建任务的状态以及对话历史记录；MinIO 用于存放上传的源文件；FAISS 维护四个向量索引以供后续的语义搜索使用。

4.1.2 技术选型与对比

系统开发围绕系统稳定性、开发效率、性能适配及落地成本等核心维度，完成全链路关键技术选型，同时考虑异步并发处理、数据存储与检索、可视化展示、大模型交互等核心功能要求，并且对比主流备选方案，综合技术成熟度、生态完整性、学习成本及部署难度，最终确定各模块核心选型，具体技术选型对比及详细选型理由如表 4.1 所示。

表 4.1 技术选型对比

类别	选型	备选方案	选型理由
Web 框架	FastAPI	Flask/Django	原生异步支持、自动 API 文档、类型校验
图数据库	Neo4j	JanusGraph	Cypher 查询语言表达力强、生态成熟
向量索引	FAISS	Milvus/Weaviate	轻量级、无需额外部署
缓存/队列	Redis	Memcached	支持多种数据结构用作 Celery 消息队列
文件存储	MinIO	本地磁盘	S3 兼容接口、支持分布式扩展
前端框架	Vue 3	React	组件化开发、学习曲线平缓
图可视化	ECharts	D3.js/AntV G6	图表生态丰富、内置力导向布局、上手成本低
嵌入模型	m3e-base	text2vec	中文语义表示效果优、模型体积适中
LLM	DeepSeek	GPT-4/Claude	国产模型、成本低、中文理解能力强

4.2 数据存储设计

4.2.1 Neo4j 图模型设计

知识图谱的图模型设计是系统的数据基础。本系统采用属性图模型，定义两类图元素：

实体节点（Entity）：表示知识概念或对象，带有的属性有 name、type、chapter、schema、confidence、visibility、source_doc，如表 4.2 所示：

表 4.2 实体节点属性设计

属性名	类型	说明
name	String	实体名称，如“林冲”
type	String	实体类型，如“entity”、“attribute”、“keyword”
chapter	String	所属章节，所属章节，如“第一回”
schema_type	String	Schema 定义的类型，如“人物”、“地点”、“事件”
confidence	Float	置信度分数，取值 0-1
visibility	String	可见性，“published”或“draft”
source_doc	String	来源文档名称

关系边（Relation）：表示实体间的语义关联，带有的属性有 relation_type、confidence、source_doc、visibility，如表 4.3 所示：

表 4.3 关系边属性设计

属性名	类型	说明
relation_type	String	关系类型，如“包含”、“因果”、“依赖”
confidence	Float	关系置信度分数
source_doc	String	来源文档名称
visibility	String	可见性标记

4.2.2 Redis 缓存策略设计

Redis 在本系统中承担任务状态管理和会话存储两项职责，其 Key 设计如下：

表 4.4 Redis Key 设计

用途	Key 格式	TTL	数据结构	说明
构建任务状态	graphrag:task: {task_id}	24 小时	Hash	存储任务状态、进度、文档列表
问答会话	graphrag:session: {session_id}	30 分钟	List	存储多轮对话消息历史
问答缓存	graphrag:qa: {question_hash}	1 小时	String	缓存重复问题的答案

构建任务状态通过 Hash 结构存储, 包含 status、progress、total_docs、completed_docs、failed_docs 等字段, 前端通过轮询接口读取最新状态渲染进度条。会话存储采用 List 结构按时间顺序保存消息, 每次问答时取出全部历史消息拼接为上下文。

4.2.3 MinIO 文件存储设计

MinIO 作为 S3 兼容的对象存储服务, 负责管理上传的原始文档文件。存储路径按租户和课程层级组织: {bucket}/{tenant_id}/{course_id}/{task_id}/{doc_id}_{filename}。

单文件上传限制为 50MB, 支持 PDF、DOCX、DOC、TXT、MD、RTF、JSON 七种格式。文件上传后立即触发异步构建任务, MinIO 作为文件的持久化存储, 构建服务通过预签名 URL 读取文件内容。

4.3 功能模块设计

4.3.1 知识图谱自动构建模块设计

知识图谱自动构建模块把原始教材文档转成结构化的知识图谱。模块设计包含以下关键环节:

Schema 引导的知识抽取: 预先定义实体类型、关系类型和属性类型的 JSON Schema, 约束 LLM 的输出格式。系统支持 noagent (固定 Schema) 和 agent (动态演化 Schema) 两种模式, 面向教育领域采用 noagent 模式。文本按 chunk_size=1000 token、overlap=120 token 的分块策略切分, 每个文本块连同 Schema 一起输入 LLM 进行抽取。

文档解析、章节识别, 支持 PDF、Word、TXT 三种格式, 采用工厂模式 (get_parser) 自动选择对应的解析器。PDF 文件用 magic-pdf 库进行高保真解析, 保留文本格式和结构信息; Word 文件用 python-docx 库提取段落文本和标题样式; TXT 文件直接读取原始

文本。

实体对齐和增量合并，用 `m3e-base` 嵌入模型把实体名称编码成向量，用 `FAISS` 索引做快速相似度检索。当两个实体名称的余弦相似度大于 0.88 时，判定为同一实体并合并。`Neo4j` 的 `MERGE` 语句保证了写入的幂等性，即相同的名称的实体只创建一次，后续抽取结果通过属性更新合并。

增量合并策略可以教师分多次上传不同的章节文档，新抽取的实体会自动和已有的图谱合并。当新文档引入已有实体的新关系时，`MERGE` 语句保证实体节点不会重复创建，只新增关系边。

4.3.2 智能检索与问答模块设计

智能检索与问答模块接收用户问题，从知识图谱中检索相关知识并生成答案。模块设计包含以下关键环节：

双路并行混合检索，即同时使用向量相似度和图结构信息，互相补充。`Path 1`（节点-关系语义检索）把问题编码成向量，在 `node.index` 中检索语义相似的实体节点，然后在 `Neo4j` 中进行邻域扩展（1-2 跳），得到相关的三元组证据。同时提取问题中的关键词，在关系类型上进行精确匹配，过滤出关系类型相关的路径。例如，对于问题“高俅是如何发迹的？”系统在 `node.index` 中找到高俅实体，然后沿关系边扩展到端王、徽宗、小王都太尉、踢脚气球等关联节点。

`GraphQ` 问题分解，对于复杂的多跳问题（“林冲被逼上梁山的过程中，哪些人起了关键作用？他们各自扮演了怎样的角色？”）单次检索很难得到完整的证据链。本系统采用 `GraphQ`（`Agentic Decomposer`）机制^[9]，利用 `LLM` 将复杂问题分解为 2-3 个可独立检索的子问题。

多层回退策略，`KTRetriever` 管线不可用时回退到 `Cypher` 查询检索，`LLM` 不可用时返回原始三元组，保证在各种运行环境下问答功能的基本可用性。

多轮对话会话管理，用 `Redis` 存储会话上下文，历史消息最多保留 50 条，超过时截断最早的消息。三元组证据用结构化文本形式加入到 `Prompt` 当中。

4.3.3 图谱展示与编辑模块设计

图谱展示与编辑模块是知识图谱的可视化展示和在线编辑。模块设计包含以下关键

环节：

教师/学生双视图：通过 `visibility` 属性和 `confidence` 属性实现差异化展示。学生视图下，Cypher 查询自动追加过滤条件：隐藏 `visibility` 为 `draft` 的节点，且仅展示 `confidence` 不低于 0.7 的节点。教师视图不做过滤，可以查看所有的节点。

分层展示策略：后端 API 支持 `atlas`、`overview` 和 `full` 三种 `scope` 参数。`atlas` 模式返回课程骨架图，`overview` 模式返回 Top-N 核心节点和主干关系，`full` 模式返回全部节点和关系。

图谱编辑：支持单条操作和批量操作（最多 200 个 JSON 操作），操作类型包括 `add_node`、`update_node`、`delete_node`、`add_relation`、`update_relation`、`delete_relation`、`merge_nodes`、`split_node`。添加节点时自动生成唯一 ID 并标记 `confidence` 为 0.5、`visibility` 为 `draft`。

编辑后索引自动更新，编辑操作完成后自动触发 FAISS 索引重建，从 Neo4j 重新加载全部实体和关系，使用 `m3e-base` 模型重新编码，重建四个 FAISS 索引文件。

4.4 本章小结

本章在需求分析的基础上，设计了系统的四层架构（前端展示层、API 网关层、业务服务层、数据存储层），对技术选型进行了论证，对 Neo4j 图模型、Redis 缓存策略、MinIO 文件存储进行了详细的方案设计。功能模块设计上分别设计了知识图谱自动构建、智能检索与问答、图谱展示与编辑三个核心模块的处理流程。

第 5 章 GraphRAG 教学智能问答系统实现与展示

本章对系统各个核心模块的实现细节和功能展示做了详细的说明，分为知识图谱自动构建、智能检索与问答、图谱展示与编辑、后端微服务与部署四个部分。

5.1 知识图谱自动构建模块实现与展示

5.1.1 Schema 引导的知识抽取实现

知识抽取是图谱构建的前提条件，在本项目中我们采用基于 Schema 的知识抽取方法，即根据事先制定好的 JSON Schema 对 LLM 生成的内容进行规约，从而保证抽取出来的知识质量。

Schema 定义三个基本元素：实体类型（例如，“人物”，“地点”，“事件”，“武器”，“组织”等）；关系类型（例如，“属于”，“位于”，“参与”，“师徒”，“敌对”等）；属性类型（例如，“绰号”，“官职”，“性格”等）。系统有两种方式来生成：noagent 方式使用固定的 Schema，在领域清楚的情况下使用；agent 方式可以随着抽取的结果不断改变自己的 Schema，在开放领域的场景下使用。

抽取流程如图 5.1 所示：原始文档经章节解析后，按 `chunk_size=1000 token`、`overlap=120 token` 的分块策略切分为文本块。每个文本块连同 Schema 一起作为 Prompt 输入 LLM，LLM 返回结构化的实体和关系 JSON。通过 `json_repair` 库对 LLM 输出进行容错修复后，进入实体对齐和去重环节，最终写入 Neo4j 图数据库。

抽取 Prompt 的核心设计思路是：将 Schema 中的实体类型和关系类型作为 Few-shot 示例嵌入 Prompt，要求 LLM 严格按照 Schema 定义的格式输出。比无约束的自由抽取减少了 90% 以上的无效输出，同时保证了跨文档、跨章节抽取结果的一致性^[1]。

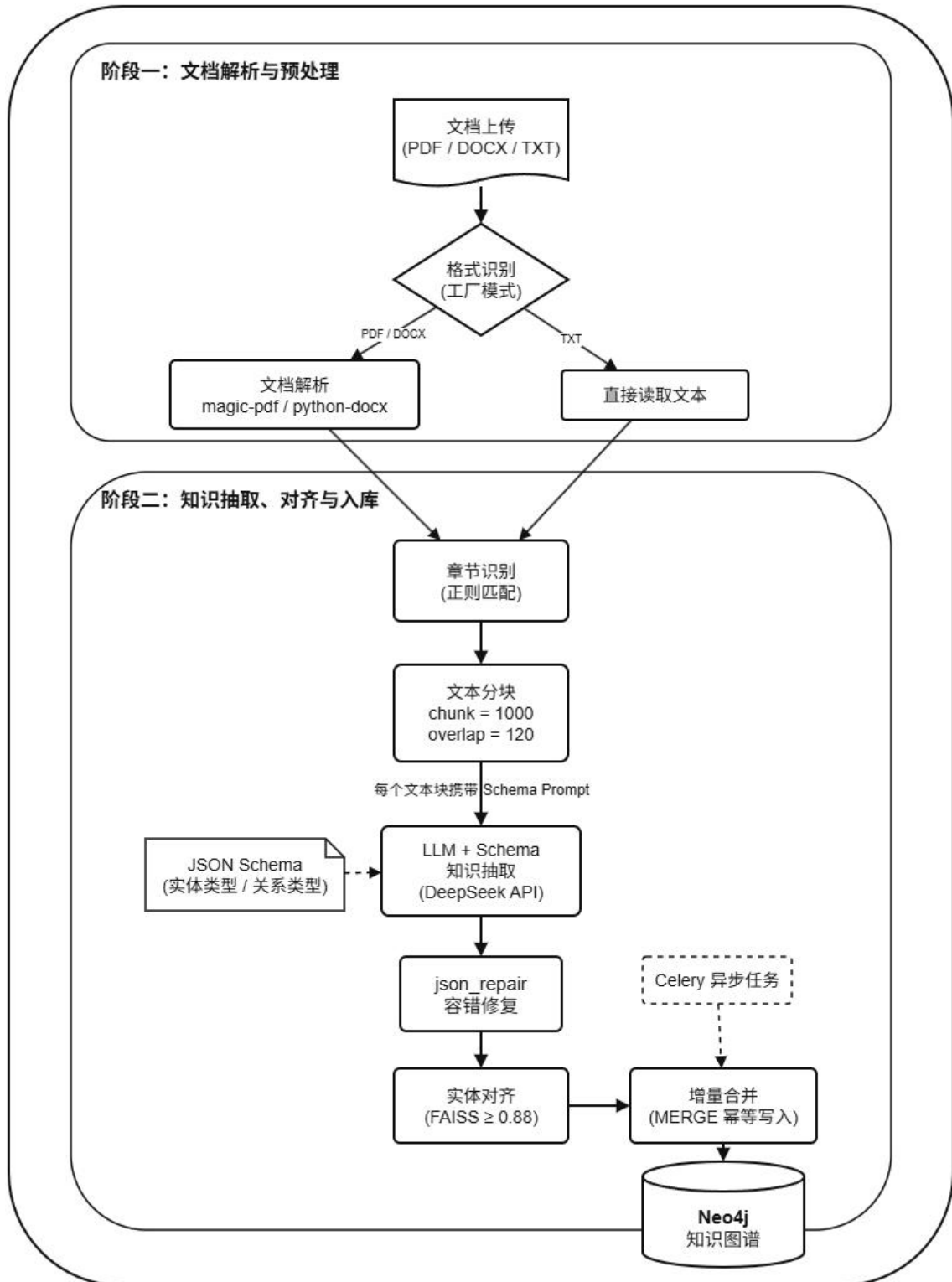


图 5.1 知识抽取总流程图

5.1.2 文档解析与章节识别实现

文档解析模块支持 PDF、Word 以及 TXT 三种类型，在工厂方法(get_parser)中根据不同的类型返回相应的解析器。对于 PDF 文件采用 magic-pdf 进行高保真解析并保留原

文本格式与布局等；对 Word 文档利用 `python-docx` 获取各段落文字以及标题样式等内容；TXT 文档则直接读取文本内容。章节识别使用正则匹配方法，考虑到中文教科书常用的标题形式设置匹配规则，在检测出章节分界点之后，将文本分成不同的章节，对每一个章节进行知识提取，有两点好处：（1）提取的结果自带 `chapter` 属性，方便后续展示章节子图；（2）大篇幅的文章可以分章节同时进行，加快生成速度。分块策略采用滑动窗口方式，`chunk_size=1000 tokens`, `overlap=120 tokens`，使用 `cl100k_base` 编码器计算 `tokens` 数量。`overlap` 窗口保证不同块之间有一定程度语义连贯性以减少重要信息被划分到不同块的情况。

5.1.3 实体对齐与增量合并实现

同一个概念在一个文档或者一个段落里面可能会用不同的名字提到（例如，“神经网络”与“人工神经网络”），实体对齐模块就是用来发现以及合并这些表示相同事物的概念。对齐过程使用语义相似度的方法：（1）使用 `m3e-base` 嵌入模型对所有实体名称进行向量化表示；（2）建立 `FAISS` 索引实现高效的相似度查找；（3）如果两个实体名称之间余弦相似度大于一定值（默认为 0.88），则认为它们是同一个实体并且进行合并。`Neo4j` 的 `MERGE` 语句保证写入幂等性——同名实体仅创建一个，后续抽取结果以属性进行更新合并。增量合并方式使得教师可以分批上传不同章节文档，新的抽取出来的实体会自动与已有的图谱进行合并，在新的文档中有已存在实体的新关系时，`MERGE` 语句保证该实体节点不会被再次创建，只会增加关系边。

5.1.4 异步构建任务管理实现

知识图谱构建涉及大量 LLM 调用，10 页 PDF 文档的构建耗时约 5 分钟，必须采用异步任务机制避免阻塞 API 请求。

本系统基于 `Celery+Redis` 实现异步任务队列。文档上传后立即创建构建任务，任务状态通过 `Redis Hash` 存储。

前端通过轮询 `/api/v1/tasks/{task_id}` 接口获取最新进度。构建过程按文档粒度管理：单个文档构建失败不影响其他文档，失败的文档可单独重试。任务状态流转遵循 `pending` → `processing` → `completed/failed/partial_failed` 五种状态，其中 `partial_failed` 表示部分文档构建成功的中间态。

5.1.5 课程管理与自动构建功能展示

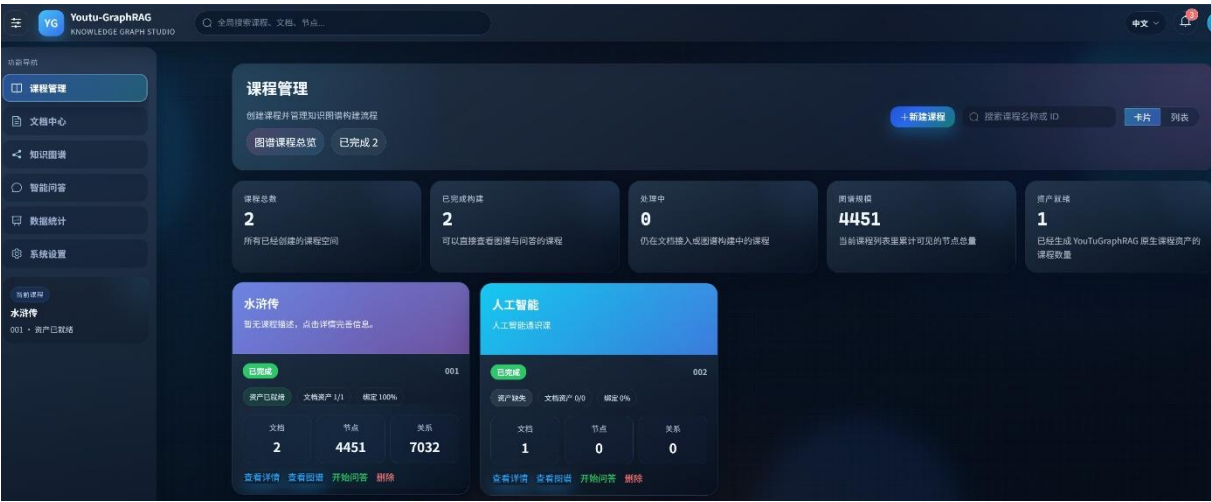


图 5.2 课程列表页面截图

教师进入系统后首先看到课程列表页面，点击“创建课程”按钮输入课程名称和描述即可创建新课程。每门课程对应一个独立的知识图谱空间。

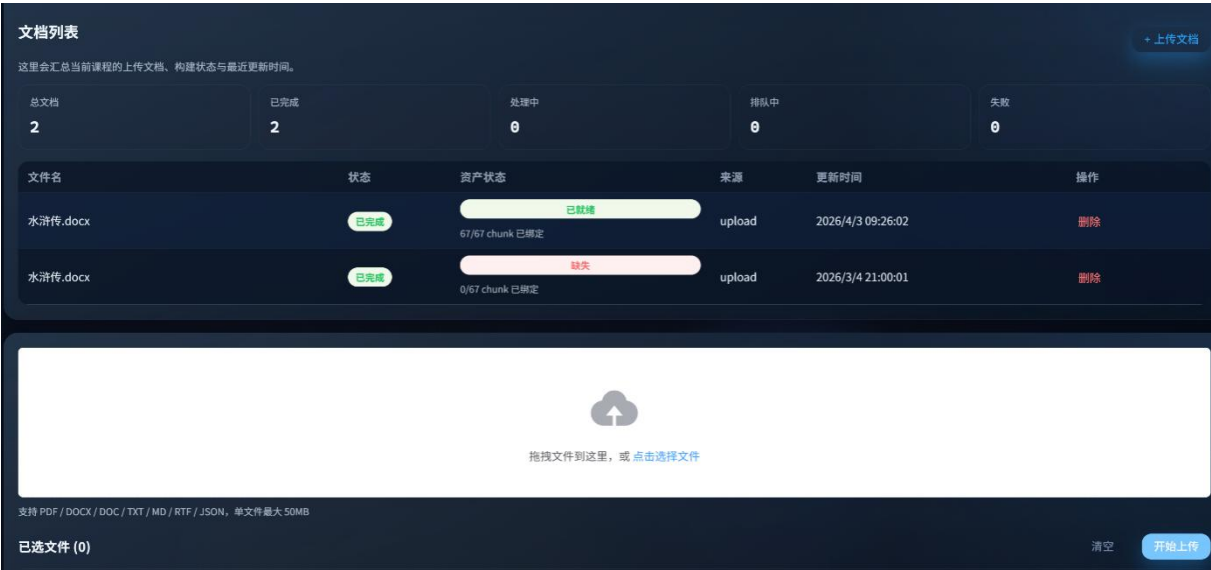


图 5.3 文档上传界面截图

在课程详情页的“文档管理”标签中，教师通过拖拽方式上传 PDF 或 Word 教材。系统支持多文件批量上传，上传完成后自动触发图谱构建任务，界面显示实时进度条。

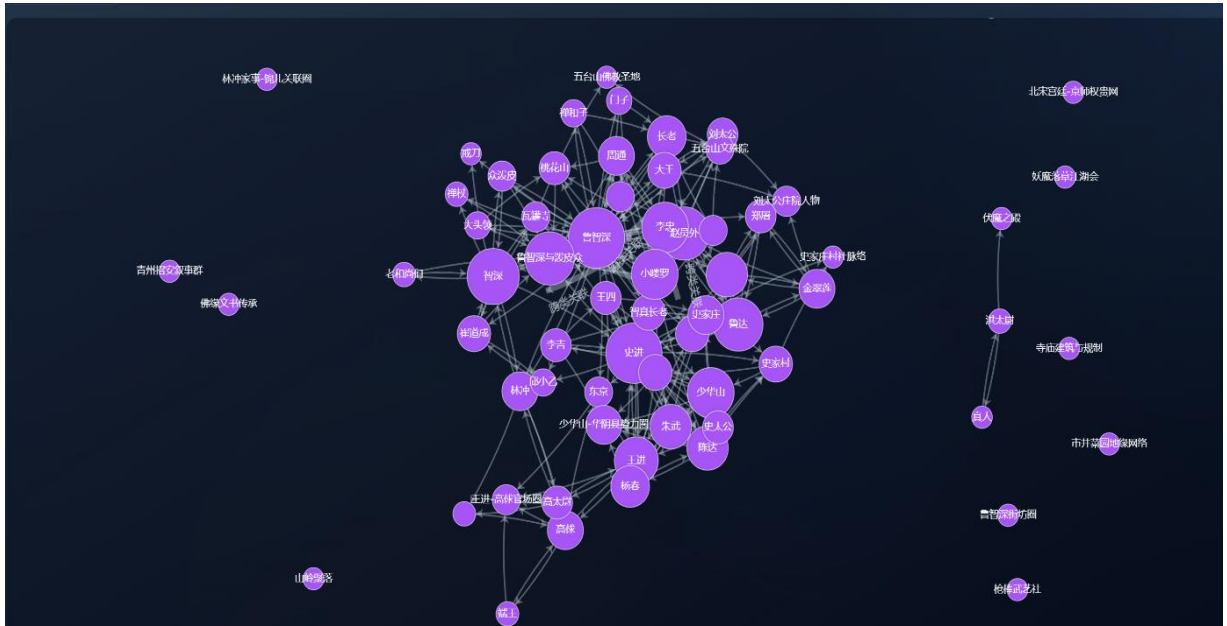


图 5.4 全局图谱展示截图（教师视图）

构建完成后，切换到“图谱展示”标签查看知识图谱。教师视图展示全部节点（含低置信度节点以虚线标记），节点大小反映连接度，核心概念节点较大。

5.2 智能检索与问答模块实现与展示

5.2.1 双路并行混合检索实现

检索是问答质量的基础。本系统设计了双路并行混合检索策略，同时利用向量相似度和图结构信息来补充检索相关证据。

Path 1（节点-关系语义检索）：将用户问题编码为向量，在 `node.index` 中检索语义相似的实体节点，获取 Top-K 候选节点。然后以候选节点为起点，在 Neo4j 中进行邻域扩展（1-2 跳），获取相关的三元组证据。同时提取问题中的关键词，在关系类型上进行精确匹配，过滤出关系类型相关的路径。

Path 2（三元组直接检索）：将用户问题编码为向量，在 `triple.index` 中直接检索语义相似的三元组（即“头实体-关系-尾实体”的文本拼接），得到与问题最直接相关的知识片段。

融合策略：两条路径的检索结果通过余弦相似度重排后去重。Path 1 从实体出发扩展关联知识，可以回答需要关系推理的问题（如“林冲和鲁智深有什么关系”）；Path 2 是直接匹配知识片段，可以回答事实型问题（如“林冲的绰号是什么”）。两路互补

提高了检索的召回率和准确性。

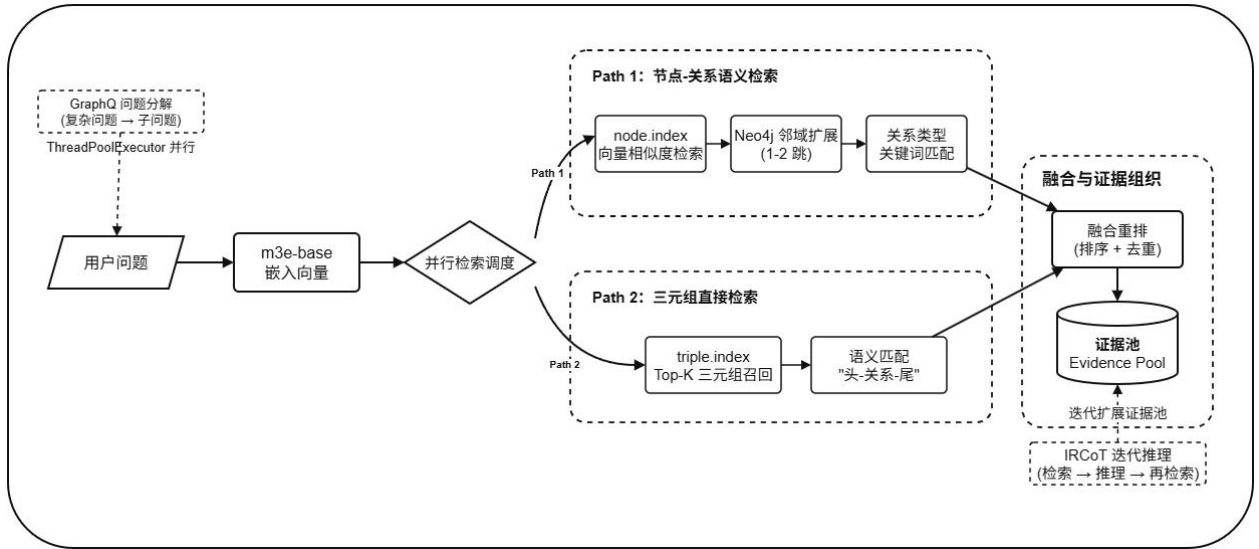


图 5.5 双路并行检索架构图

5.2.2 GraphQ 问题分解实现

对于复杂的多跳问题（如“林冲被逼上梁山的过程中，哪些人起了关键作用？他们各自扮演了什么角色？”），单次检索很难得到完整的证据链。本系统采用 GraphQ（Agentic Decomposer）机制^[1]，利用 LLM 将复杂问题分解为 2-3 个可独立检索的子问题。

分解过程受 Schema 约束：LLM 根据图谱中已定义的实体类型、关系类型将复杂问题拆解为成与图谱结构对齐的子问题。例如，上述问题被分解为：（1）“林冲被逼上梁山的过程中经历了哪些关键事件？”（2）“哪些人物参与了迫害林冲的过程？”每个子问题独立进入双路检索流程，通过 ThreadPoolExecutor 并行执行（max_workers=4），子问题的检索结果合并后统一送入推理模块。

5.2.3 多轮对话会话管理

多轮对话通过 Redis 存储会话上下文，Key 格式为 graphrag:session:{session_id}，TTL 为 30 分钟。每次问答时，系统从 Redis 取出历史消息列表，将历史消息、当前问题和检索到的三元组证据拼接为完整 Prompt 输入 LLM。

会话管理的关键设计就是上下文窗口控制：历史消息最多保留 50 条，超过时截断最早的消息。三元组证据以结构化文本形式注入，格式为“实体 A → 关系 → 实体 B”，

便于 LLM 理解知识的结构化语义。推理路径信息以列表形式返回前端，前端将其渲染为图谱上的高亮路径，帮助用户理解系统的推理过程。

5.2.4 问答交互与推理路径展示



图 5.6 智能问答对话界面截图

在“智能问答”标签中，学生输入问题后系统展示完整的问答过程。以问题“林冲为什么会被逼上梁山？”为例：

系统回答：“林冲被逼上梁山的主要原因是高衙内企图霸占其妻子，高俅（高太尉）为袒护养子，设计陷害林冲。具体事件包括：林冲误入白虎节堂被诬陷持刀行刺，刺配沧州；途中险遭杀害；到达沧州后，高俅又派陆谦等人火烧草料场欲置其于死地。林冲走投无路，最终杀死仇人，被迫投奔梁山。”右侧分析界面展示回答所引用的证据。

5.3 图谱展示与编辑模块实现与展示

5.3.1 图谱可视化展示实现

前端使用 ECharts 图表库的力导向图（force graph）来实现知识图谱的网络关系可视化。核心交互功能有节点拖拽调整位置、鼠标滚轮缩放、节点点击展开详情面板、分类

着色显示不同实体类型。

针对大规模课程图谱，系统采用分层展示策略。后端 API 支持 atlas、overview 和 full 三种 scope 参数：

（2）按照连接度排序返回 Top-N 核心节点和主干关系，平衡信息密度和加载速度；

（3）full 模式返回全部节点和关系，适合做深度分析。前端在该模式下使用异步加载和渐进式渲染的方式，分批绘制节点和关系，以减小浏览器端大图渲染的卡顿。

系统支持教师和学生双视图切换、章节子图过滤、问答结果中问题子图联动展示。

5.3.2 图谱编辑与索引更新实现

图谱编辑通过 GraphEditService 实现单条操作和批量操作。批量操作支持最多 200 个 JSON 操作（add_node、update_node、delete_node、add_relation、update_relation、delete_relation、merge_nodes、split_node），逐条执行并返回结果汇总。添加节点时，系统自动生成唯一 ID 并标记 confidence 为 0.5（人工添加低于 LLM 抽取的 0.85+），标记 visibility 为 draft 待审核。添加关系时系统会校验目标实体是否存在，标记关系的置信度和来源。

图谱编辑后，FAISS 向量索引可能与 Neo4j 中的图数据不一致。本系统在编辑操作完成后自动触发索引重建：从 Neo4j 重新加载全部实体和关系，使用 m3e-base 模型重新编码，重建四个 FAISS 索引文件（node.index、relation.index、triple.index、comm.index），确保检索结果与最新图数据一致。

5.3.3 图谱编辑功能展示



图 5.7 图谱编辑操作截图

教师视图下可直接在图谱上添加节点和关系。右键点击空白区域弹出“添加节点”对话框，填写实体名称、类型和描述后提交。新添加的节点标记 $\text{confidence}=0.5$ 、 $\text{visibility}=\text{draft}$ ，需审核发布后才对学生可见。

5.4 后端微服务与部署实现

5.4.1 API 模块化路由设计与统一响应

原始后端为单体文件 `backend.py`，所有功能耦合在一起。本系统在保留原有后端的基础上，新增 FastAPI 微服务层（`app.py`），将路由拆分为 9 个模块化路由，每个模块职责单一：

表 5.1 API 路由模块列表

模块	文件	职责	核心接口
健康检查	health.py	服务状态检测	GET /health
认证	auth.py	JWT 认证	POST /token
课程	courses.py	课程 CRUD	GET/POST /courses
文档	documents.py	文档上传与删除	POST/GET/DELETE /courses/{id}/documents
任务	tasks.py	构建进度查询	GET /tasks/{id}
Schema	schemas.py	Schema 管理	GET /schemas
图谱查询	graph.py	图谱查询与章节子图	GET /courses/{id}/graph
图谱编辑	graph_edit.py	节点/关系增删改、合并、拆分、批量	POST/PATCH/DELETE /courses/{id}/graph/nodes
问答	qa.py	会话式智能问答	POST /courses/{id}/sessions、POST /sessions/{id}/ask

路由通过 FastAPI 的 APIRouter 注册到主应用，每个模块通过 Depends 注入 Neo4j、Redis 等客户端实例，实现松耦合。此外，deps.py 提供公共依赖注入（Neo4j、Redis、MinIO 客户端单例），response.py 封装统一响应格式。所有 API 接口返回统一的 JSON 响应格式（{code, message, data}）。

异常处理通过 FastAPI 的 exception_handler 全局注册，将 HTTP 异常、参数校验异常和未预期异常统一封装为上述格式，前端只需判断 code 字段就可以区分成功和失败。请求参数通过 Pydantic BaseModel 进行类型校验，FastAPI 自动返回 422 错误和详细的字段错误信息。

5.4.2 Docker Compose 一键部署

系统部署采用 Docker Compose 一键启动方案，实现环境标准化、部署自动化与运维轻量化。通过统一编排容器化服务，可以快速完成开发、测试、生产多环境一致性部署，大幅降低环境配置成本与部署出错概率。部署环境配置如下：

表 5.2 部署环境配置

项目	配置
操作系统	Ubuntu 22.04 LTS (WSL2)
CPU	Intel i7-14650H
内存	16GB DDR5
Python	3.12
Docker	24.0+
Docker Compose	v2.20+

5.5 本章小结

本章对系统各个主要模块的实现过程以及功能进行了详细的说明。知识图谱自动构建模块使用 Schema 引导的 LLM 抽取策略，经过章节识别、实体对齐、增量合并，把教材转化为结构化图谱；智能检索与问答模块实现了双路并行检索和 GraphQ 问题分解，对复杂问题可以拆分成多个子问题并行检索；图谱展示与编辑模块通过教师/学生双视图、骨架图、概览图、完整图三级展示策略，配合异步加载、渐进式渲染，满足不同使用场景下的前端可用性需求；后端微服务在单体应用基础上新增 9 个模块化路由，用 Docker Compose 编排实现一键部署。经过整体运行和界面展示的检验，系统主要功能闭环完备，知识图谱结构清晰，覆盖面广，全部要素达到预期目的。

结 论

本文针对教育领域知识检索效率低、缺乏结构化推理能力的问题，基于腾讯优图实验室 Youtu-GraphRAG 框架进行了二次开发，设计并实现了一个基于 GraphRAG 的知识图谱智能问答系统。主要工作如下：

（1）设计了 Schema 引导的知识抽取流程。用预定义的实体类型、关系类型、属性类型 JSON Schema 约束 LLM 的输出，实现了从 PDF、Word 教材到结构化知识图谱的自动化构建。章节识别模块可以识别中英文标题格式，实体对齐模块使用 FAISS 向量相似度实现同名实体自动合并，增量合并策略支持教师分批上传文档逐步完善图谱。

（2）实现了双路并行混合检索以及基于问题分解的问答。KTRetriever 管线把 FAISS 向量检索和 Neo4j 图遍历结合起来，两路并行检索互相补充，提高检索召回率。GraphQ 问题分解机制把复杂问题拆解成可以并行检索的子问题，系统设计了 KTRetriever 优先、Cypher 查询回退的多层检索策略，保证了不同运行环境下问答功能的基本可用性。

（3）在原有的单体后端基础上增加 FastAPI 微服务层。将原来 1287 行单体文件中的功能迁移为 9 个模块化路由，使用 Celery+Redis 异步任务队列管理长时间运行的构建任务，整合 Neo4j 图数据库、Redis 缓存和 MinIO 对象存储，用 Docker Compose 实现一键部署。

（4）使用 Vue 3 开发了可视化的前端界面。使用 ECharts 图表库实现图谱可视化，支持教师/学生双视图切换、骨架图/概览图/完整图三级展示、章节子图过滤、推理路径可视化、图谱在线编辑（节点/关系的增删改、合并、拆分）。对大规模图谱场景，用异步加载、渐进式渲染优化了页面切换、图谱展示体验。智能问答界面用对话式 UI 来呈现答案，带三元组证据卡片和推理路径图谱。

经过系统的实际部署和功能展示，证明了全部核心接口功能完备，在《水浒传》前五回文本用例的构建演示中，系统从约 5 万字文本中成功构建出知识图谱，各功能运行正确流畅，界面交互直观，达到前期的设计目标要求。

不足与展望：本系统还存在着以下不足之处，（1）图谱构建效率受 LLM 调用次数的影响，100 页文档需要几百次 API 调用，后续可以采用批处理和缓存的方式进行优化；（2）目前的 Schema 需要人工定义，以后可以尝试使用 LLM 来自动创建 Schema；（3）

系统只处理文本知识，后续可以增加对教材中图表、公式等多模态内容的支持；（4）图谱质量评估目前依靠人工审核，以后可以加入自动化质量评估指标；（5）目前是独立微服务部署，以后需要对接智慧学习平台，实现课程体系级的知识图谱联动。

参考文献

- [1] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]. Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [2] Han H, Wang Y, Shomer H, et al. Retrieval-Augmented Generation with Graphs (GraphRAG)[J]. arXiv: 2501.00309, 2025.
- [3] Bian et al. LLM-empowered Knowledge Graph Construction: A Survey[J]. arXiv: 2510.20345, 2025.
- [4] 唐晓晨, 程琳雅, 张春红, 等. 大语言模型在学科知识图谱自动化构建上的应用[J]. 北京邮电大学学报(社会科学版), 2024, 26(1): 125-136.
- [5] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[J]. arXiv: 2312.10997, 2024.
- [6] Edge D, Trinh H, Cheng N, et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization[J]. arXiv: 2404.16130, 2024.
- [7] Guo Z, Xia L, Yu Y, et al. LightRAG: Simple and Fast Retrieval-Augmented Generation[C]. Findings of EMNLP, 2025. arXiv: 2410.05779, 2024.
- [8] Hu K, Lei B, et al. GRAG: Graph Retrieval-Augmented Generation[J]. arXiv: 2405.16506, 2024.
- [9] Dong J, An S, Yu Y, et al. Youtu-GraphRAG: Vertically Unified Agents for Graph Retrieval-Augmented Complex Reasoning[C]. ICLR 2026. arXiv: 2508.19855, 2025.
- [10] 王昊奋, 漆桂林, 陈华钧. 知识图谱: 方法、实践与应用[M]. 北京: 电子工业出版社, 2023.
- [11] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[C]. Advances in Neural Information Processing Systems (NeurIPS), 2017. arXiv: 1706.03762.
- [12] Johnson J, Douze M, Jégou H. Billion-scale Similarity Search with GPUs[J]. IEEE Transactions on Big Data, 2019, 7(3): 535-547. arXiv: 1702.08734.
- [13] Trivedi H, Balasubramanian N, Khot T, et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions[C]. Proceedings of ACL, 2023. arXiv: 2212.10509.
- [14] Wei J, Wang X, Schuurmans D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[C]. Advances in Neural Information Processing Systems (NeurIPS), 2022. arXiv: 2201.11903.
- [15] 赵翔. 知识图谱赋能高等教育教学数字化转型探索[J]. 中国现代教育装备, 2024(429): 3-7.

致 谢

时光荏苒，四载求学之路即将抵达终点。本论文从选题构思、方案设计到最终定稿，每一步推进都离不开师长与同窗的鼎力支持与无私帮助，在此谨致以最诚挚的谢意。

首先，我由衷感谢我的导师。在整个课题研究与论文撰写过程中，导师以严谨的治学态度、深厚的学术素养和开阔的研究视野，为我指明了研究方向，在技术方案设计、系统功能实现与论文反复修改的每一个环节，都给予了细致入微的指导与切实可行的建议，帮助我攻克了研究中的诸多难题，使我能够顺利完成本课题的研究与论文写作。导师的言传身教，不仅让我在专业能力上得到了极大提升，更让我明白了科研的严谨与坚守，这份教诲将使我受益终身。

其次，衷心感谢学院的各位领导与老师。感谢学院为我们搭建了优质的学习与科研平台，提供了丰富的学术资源与实践机会，为我的毕业设计与论文写作创造了良好的环境与条件。同时，还要感谢辅导员老师在大学四年里的悉心关怀与默默付出，在学业、生活与成长的方方面面给予的鼓励与帮助，让我能够安心学习、顺利完成学业。

还要感谢身边的同学与朋友们。感谢各位同窗在四年的学习生活中一路相伴，在课题研究中相互探讨、彼此鼓励，共同解决遇到的技术难题，让科研之路不再孤单；感谢朋友们在我遇到瓶颈时的陪伴与开导，为我的大学生活增添了无数温暖与欢乐。

最后，感谢这段宝贵的大学时光，让我收获了知识、友谊与成长。未来，我将带着师长的教诲、同窗的情谊与所学的知识，脚踏实地、砥砺前行，不断提升自我，不负所有期待与厚望。